

Computer Vision with Modern Convolution Neural Networks

Arwyn Thandrayen
23759706
23759706@stu.mmu.ac.uk

I. INTRODUCTION

We have seen rapid advancements in the ability of computers to generate realistic fake human images using neural networks. Generative adversarial networks (GANs), such as StyleGAN, use neural networks to create deep-fake images that are increasingly challenging to distinguish from real ones. These fake images can be used for nefarious reasons, to mislead and create distress. Thus, the need for robust and accurate models that can detect these images is vital. Convolution neural network (CNN) models such as AlexNet were developed to process and learn from visual data. However, as deep fake images have become more sophisticated, with higher resolutions, the limitations of these models have been exposed. The solution has been modern CNN's such as ResNet18.

The aim of this project is to demonstrate an understanding of how modern CNN models such as ResNet18 can be used to distinguish between AI generated images of human faces and real ones. We will examine some of the theory and architecture behind these models as well as conduct experiments by inputting a mixture of real and fake jpeg images from a dataset and outputting the classification of real or fake. We will also analyse metrics such as accuracy and loss to assess model its proficiency.

II. DATASET

The dataset we are using is called *StyleGan-StyleGan2 Deepfake Face Images* owned by Kshitiz Bhargava and available on Kaggle.com. This dataset has been created by merging and augmenting two existing datasets, containing images of faces. The real images both come from photography websites, Nvidia's Flickr and Unsplash. The fake ones have been created by StyleGAN and StyleGAN2. Combining these datasets aims to ensure that the model trained on this data works well across different domains.

The dataset has already been augmented through techniques such as rotation, shifting, brightness modification and flipping with the aim of improving model robustness. In total we have 12,890 images, 5,890 are real and represent a range of characteristics including age and sex. 7,000 are fake. Our Final Dataset folder size is 257.6 MB and has already been split into Fake and Real sub-folders containing .jpg images.

Table.1. Dataset Images

Face Images	Real	Fake	Total
Number	5890	7000	12890

III. METHODOLOGY

A. Model

We upload our dataset. It is already split into categories of Real and Fake. We next split the dataset for purposes of testing and training in a ratio of 3:7. So 70 % of our data will be used to train the model and 30 % to test it. We use PyTorch framework for this model. We perform our own dynamic data augmentation. This applies random changes to the training samples without expanding the size of the dataset. To the training data we create diverse inputs by altering the colour brightness, contrast and saturation. We flip and crop images and process them for ease of use with our model including re-sizing to 224x224 pixels. We don't need to augment our test data but we do apply steps such as resizing and scaling for purposes of consistency. We then apply torchvision to transform the data into subfolders for the train, test and validation data. The validation data is used to fine tune the model. We split the train data for mini batch loading with 20% for validation. Our batch size is 32. Feeding the data in mini-batches helps with efficiency and memory management of the model. Our model will be a classifier since it has two classes or outputs, Real and Fake. On examination we have 7218 training samples, 1905 for validation and 3867 for testing.

Table.2. Train/Test/Validation split

Samples for training	7218
Samples for validation	1805
Samples for testing	3867
Total samples	12890



Fig.1. Data Visualization and Augmentation

Modern CNN's such as ResNet18 have a more complex architecture with multiple layers and use batch normalization and residual connections to improve training stability and efficiency. ResNet18(weights='IMAGENET1K_V1') has been pretrained on pictures from the ImageNet database which is why we resize our images. This is called transfer learning. This means our model be building on what it has already learned not starting from scratch. It is particularly helpful if we have limited new data.

The diagram illustrates a CNN architecture for handwritten digit recognition. The process starts with an **INPUT** of size $28 \times 28 \times 1$. It passes through **Conv_1 Convolution** (3 x 5 kernel, valid padding) to produce **n1 channels** ($24 \times 24 \times n1$). This is followed by **Max-Pooling** (2×2) to produce **n2 channels** ($12 \times 12 \times n2$). Another **Conv_1 Convolution** (3 x 5 kernel, valid padding) is applied to produce **n2 channels** ($8 \times 8 \times n2$). This is followed by another **Max-Pooling** (2×2) to produce **n2 channels** ($4 \times 4 \times n2$). The final **Max-Pooling** layer is labeled **Flattened**. The output is then passed through **fc_3 Fully-Connected** (Neural Network, RELU activation) and **fc_4 Fully-Connected** (Neural Network) layers. The final output is a vector of size 10, representing the digits 0 through 9. The diagram also indicates **n2 units** and **n2 units** for the fully connected layers.

Diagram illustrating the proposed ResNet-50 architecture. The network consists of an input layer, followed by four stages of residual blocks, and a final output layer. The stages are defined by the number of residual blocks and the output feature maps:

- Stage 1: 3 residual blocks, output 64.
- Stage 2: 3 residual blocks, output 128.
- Stage 3: 3 residual blocks, output 256.
- Stage 4: 3 residual blocks, output 512.

The final output is processed by an Avg pool layer, followed by a fully connected (FC) layer, and a Softmax layer for classification.

```
(layer1): Sequential(
  (0): BasicBlock(
    (conv1): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (relu): ReLU(inplace=True)
    (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
    (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
  )
)
```

$$L(y, \hat{y}) = - \sum_{j=1}^q y_j \log \left(\frac{\exp(o_j)}{\sum_{k=1}^q \exp(o_k)} \right)$$

B. Loss

C. Optimization

Discussions

$$\theta = \theta - \eta \nabla_{\theta} J(\theta; x_i, y_i)$$

Fig.6. Residual Block and skip connection.

IV. EXPERIMENT

A. Evaluation Metrics

We are training the model on batch size of 32 and learning rate of 0.01. A larger batch size could lead to overfitting and a smaller one take longer to train the model. Batch size of 32 is a suitable level of speed and efficiency. Similarly with the learning rate of 0.01, a smaller number would take longer to train but go too large and the danger is that we converge too soon and miss subtle details. We will record the loss and percentage accuracy of our model on the training and validation data based on the model predictions run over 20 epochs which is a suitable number for a dataset of around 10,000 images. It should be enough to extract features and achieve a stable performance. As well as accuracy percentage we will record the F1 score which is a measure of the precision and recall metrics.

B. Quantitative Results

On 20 epochs with a learning rate of 0.01 and batch size of 32, using the SGD optimizer we get a high training accuracy of almost 98% and low loss rate of 0.06 with both metrics improving as the number of epochs increased and likely would continue to do so. This shows our model is performing well. The validation accuracy as expected, is lower, at 96%, which indicates the model is not overfitting. The validation loss is low at just over 0.1 and could potentially be improved. Over the 20 epochs we see from the graph that the validation has fluctuated wildly at times, although generally following the direction of the training data. This could mean the learning rate is too high. Fig.7.

C. Comparisons

We can perform hyperparameter tuning by adjusting the learning rate and batch size.. Learning rates range between 0.001 – 0.1. Using a lr of 0.001, over 20 epochs we find our accuracy and loss metrics have deteriorated. This is probably due to the slower rate of learning and since the model was improving steadily, we may need to run over more epochs. We saw less fluctuation and a more stable convergence Fig.8. We also alter the batch size to 64 with lr = 0.01. The performance was not improved when compared to using batch size 32 over 20 epochs. The model output was stable but was less adaptable due to the larger batch size and likely over generalised which negatively impacted the loss. We also compare the output of using CNN model AlexNet on our dataset. It has a less complex architecture consisting of just eight layers: five convolutional layers that extract image features, followed by three fully connected layers for classification. We see the accuracy plateau after just a few epochs and, as expected, the performance doesn't match ResNet18. See Table1 for results.

So far our optimal parameters are batch size 32 with learning rate 0.01. We visualize the performance on the test data with a heat map of a confusion matrix. We see our model had a strong performance overall with an F1 score of 0.975. It was more successful at identifying real images than fake ones.

For future investigations we could continue hyperparameter tuning, and increase the number of epochs. It would also be interesting to examine the images that were incorrectly identified and what we could learn from them. We

could also use a different optimizer such as Adam, an advanced optimization algorithm and compare results.

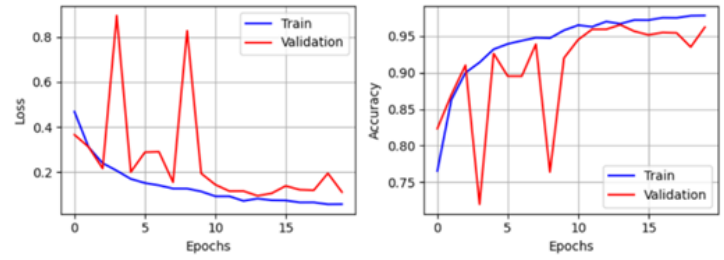


Fig.9. ResNet18 performance, lr 0.01, batch size 32,

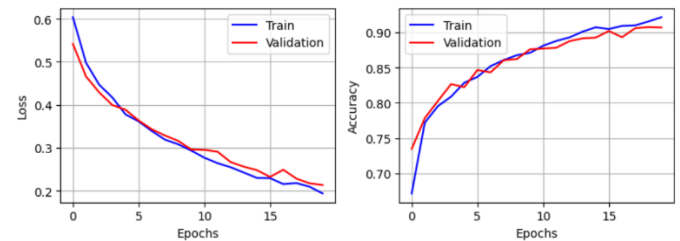


Fig.10. ResNet18 performance, lr 0.001, batch size 32,

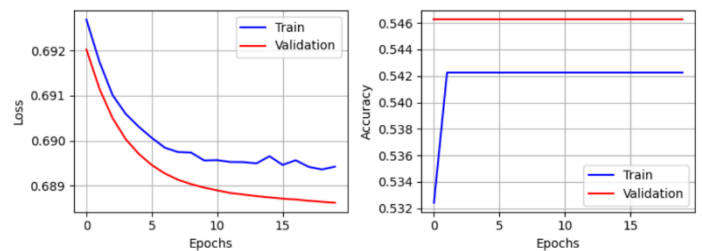


Fig.11. AlexNet18 performance, lr 0.01, batch size 32,

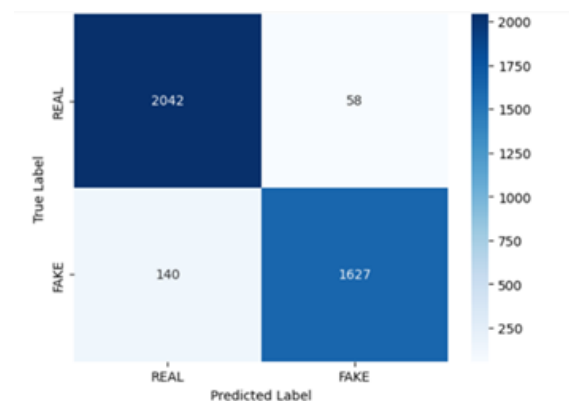


Fig.12. Heatmap of confusion matrix for ResNet18

TABLE1

Table of results including hyperparameter tuning and comparsion with AlexNet.

Model	Epochs	Batch Size	Learning Rate	Optimizer	Train		Validate	
					Loss	Accuracy	Loss	Accuracy
ResNet18	20	32	0.01	SGD	0.0587	0.9784	0.1121	0.9623
	20	32	0.001		0.1934	0.9214	0.2131	0.9070
	20	64	0.01		0.2363	0.9000	0.2670	0.8860
AlexNet	20	32	0.01	SGD	0.6894	0.5423	0.6886	0.5463